

Block Round

Математика проекта

Техническая и дидактическая документация математических основ генератора кругов, эллипсов, сфер и эллипсоидов, *визуализируемых с использованием текстур блоков Minecraft*. Для каждого понятия представлены формальное определение, геометрическое обоснование, прямое соответствие исходному коду проекта и практический перевод в команды построения внутри игры (`/fill`, `/clone`, `//sphere`).

Рассматриваемые области: аналитическая геометрия · дискретная растеризация · цифровая топология · интегральное исчисление · линейная алгебра · тригонометрия · арифметика инвентаря · наложение текстур · октаэдрическая симметрия.

Содержание

1. Обзор: математическая природа задачи	3
2. Система координат и дискретная решётка вокселей	4
3. Основное уравнение: круг и эллипс	5
4. Евклидов алгоритм (тест расстояния)	5
5. Алгоритм Брезенхэма (целочисленная средняя точка)	7
5.1. Круговой случай	7
5.2. Эллиптический случай (две области)	8
6. Пороговый алгоритм (покрытие углом)	8
7. Режимы рендеринга: Заполненный, Тонкий, Толстый	9
7.1. Заполненный	9
7.2. Тонкий (контур)	9
7.3. Толстый	10
8. Вычисление дискретной площади	10
9. От 2D к 3D: уравнение эллипсоида	11
10. Воксельзация и вычисление объёма	11
11. Геометрические срезы: плоскости и диагональный срез под 45°	12
12. Толстый режим 3D: трёхмерализация через слои	13
13. 3D-камера: сферические координаты	13
14. Авто-зум: ограничивающая сфера и FOV	14
15. Затенение и умножение текстуры	14
16. Переход: от непрерывной математики к строительству в игре	15
17. Арифметика инвентаря Minecraft	16
17.1. Одна ячейка, один стак, полный пак	16
17.2. Хранение: сундуки, двойные сундуки, шалкеры	16
17.3. Справочная таблица: канонические сферы	16
18. Подсветка обводки и подсчёт открытых граней	17
18.1. Что считается открытой гранью	17

18.2. Формула обнаружения границы	17
18.3. Подсчёт открытых граней как эвристика выживания	17
19. Послойное строительство (горизонтальное разложение)	18
20. Оптимизация через октаэдрическую симметрию (O_h)	19
20.1. Конкретная последовательность команд	19
20.2. Сравнение временных затрат	20
21. Текстуры блоков и визуальная композиция	20
21.1. Шесть граней и UV-разметка	21
21.2. Семейства блоков и тона	21
21.3. Градиенты через смещение блоков	21
22. Заключение	22
Приложение А: справочные таблицы и пошаговый пример	24

1. Обзор: математическая природа задачи

Block Round — генератор закруглённых форм, *визуализируемых текстурами блоков Minecraft*: круги и эллипсы в 2D, сферы и эллипсоиды в 3D. Центральная задача относится к классу *дискретной растеризации*: дана кривая или поверхность, аналитически определённая над $\mathbb{R}$, требуется определить, какие ячейки регулярной решётки (пиксели в 2D, воксели в 3D) должны быть отмечены для её представления. В Block Round каждая отмеченная ячейка дополнительно получает шестигранную текстуру из палитры Minecraft (блок травы, булыжник, дубовые доски, кварц, алмаз и т.д.), поэтому результирующая фигура есть не просто математический силуэт, а *строимая структура*, которую пользователь может воспроизвести в режиме выживания, в творческом режиме через `/fill`, либо импортировать через Sponge Schematic v2 (`.schem`) в WorldEdit, Litematica или MCEdit.

Задача не имеет единственного решения. Различные критерии включения порождают различные дискретные силуэты, и каждый критерий имеет своё математическое обоснование. По этой причине проект реализует три различных 2D-алгоритма (Евклидов, Брезенхэма, Пороговый), три режима рендеринга (Заполненный, Тонкий, Толстый) и три стиля 3D-затенения. Выбор алгоритма определяется не только визуальной гладкостью, но и числом блоков, которые пользователю придётся собрать: сфера диаметром $D = 20$ требует 4 189 блоков по Евклидовому алгоритму, но до 4 322 блоков по Пороговому.

Выполнение происходит целиком в браузере, без серверного компонента, что налагает дополнительное требование *вычислительной эффективности* (алмазная сфера $D = 32$ должна рендериться с частотой 60 fps, включая 17 156 текстурированных вокселей). Привлекаемые теоретические области включают:

- аналитическую геометрию коник и квадрик;
- инкрементные алгоритмы с целочисленной арифметикой (Брезенхэм);
- цифровую топологию (4-, 6- и 26-связные окрестности);
- интегральное исчисление и считающую меру;
- линейную алгебру (секущие плоскости, перспективную проекцию);
- тригонометрию и сферические координаты;
- комбинаторную арифметику инвентаря и симметрию групп.

Нововведения относительно родственного проекта Pixel Round. Block Round добавляет пять математических соображений, возникающих только когда ви-

зуализируемые ячейки становятся *блоками Minecraft*: (i) UV-разметка шести граней каждого вокселя; (ii) арифметика инвентаря выживания (паки по 64 предмета, одиночные сундуки на 27 ячеек, двойные сундуки на 54 ячейки); (iii) подсветка обводки, используемая для подсчёта открытых граней во время строительства; (iv) разложение трёхмерной формы на горизонтальные слои, так как реальное строительство ведётся этаж за этажом; (v) использование октаэдрической группы симметрии O_h для сокращения затрат на ручное размещение блоков в 8 раз через `/clone`.

2. Система координат и дискретная решётка вокселей

Холст — решётка из $G_x \times G_y$ ячеек в 2D, расширенная до $G_x \times G_y \times G_z$ вокселей в 3D. Каждая ячейка (i, j) занимает единичный квадрат $[i, i + 1] \times [j, j + 1]$; в 3D каждый воксель занимает единичный куб. В непрерывном коде **центр** ячейки находится в $(i + \frac{1}{2}, j + \frac{1}{2})$. Это соглашение критически важно: сравнение окружности с *углом* (i, j) или с *центром* изменяет, какие ячейки считаются принадлежащими форме, и, следовательно, какие блоки должен поставить пользователь.

$$\text{центр ячейки } (i, j) = (i + \frac{1}{2}, j + \frac{1}{2})$$

Для формы диаметра D код расширяет решётку до $D + 2$ ячеек по каждой оси (запас в одну ячейку с каждой стороны). Это гарантирует, что крайние пиксели (С, Ю, В, З) никогда не попадают на границу матрицы. **Центр** формы находится в $(G_x/2, G_y/2)$, а *полуоси* равны $r_x = D/2$ и $r_y = D/2$. В 3D та же логика расширяется по оси z . Единичная ячейка холста совпадает с единичным блоком Minecraft, поэтому координаты алгоритма отображаются один к одному на игровые координаты (x, y, z) .

Перенос в Minecraft. Если игрок появляется в $(0, 64, 0)$ и хочет сферу $D = 16$ ($r = 8$) на земле, центр в мировых координатах должен находиться в $(0, 72, 0)$ (то есть на 8 блоков выше поверхности), чтобы южный полюс касался пола. Соответствующая команда WorldEdit: `//sphere stone 8`, выполняемая в $(0, 72, 0)$.

3. Основное уравнение: круг и эллипс

Вся теория проекта начинается с неявного уравнения эллипса с центром в начале координат и полуосями r_x и r_y :

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

При $r_x = r_y = r$ эллипс вырождается в **окружность** радиуса r . Точки *внутри* удовлетворяют ≤ 1 ; точки *вне*, > 1 . Это и есть неравенство, определяющее, принадлежит ли воксель форме, и, следовательно, ставит ли пользователь блок в этой позиции.

Сдвигая центр в (c_x, c_y) и вычисляя в центре каждой ячейки:

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{внутри} \iff d_x^2 + d_y^2 \leq 1$$

Перенос в Minecraft. При постройке башни в выживании пользователь обычно хочет *широкий плоский круг* в основании и постепенно уменьшающиеся круги при подъёме башни. Каждый этаж — это применение неявного уравнения эллипса с теми же полуосями, но с другим z . Block Round вычисляет 2D-слой один раз, а пользователь дублирует его командой `/clone` на каждый этаж.

4. Евклидов алгоритм (тест расстояния)

Идея. Для каждой строки j аналитически найти столбцы i , лежащие внутри эллипса. Разрешая уравнение эллипса относительно x при заданном y :

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

Для строки j с вертикальным смещением $d_y = j + \frac{1}{2} - c_y$, *горизонтальная полуширина* эллипса на этой высоте равна:

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

Если $d_y^2/r_y^2 \geq 1$, строка не пересекает эллипс и пропускается. Иначе заполняются все столбцы i , такие что $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$. Целочисленные границы получаются из:

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

Обоснование. Член $+\frac{1}{2}$ возникает из соглашения, что ячейка с индексом i имеет центр в $i + \frac{1}{2}$. Функции $\lceil \cdot \rceil$ и $\lfloor \cdot \rfloor$ выполняют преобразование непрерывного интервала в целочисленные индексы, гарантируя, что включаются только ячейки, чей **центр** принадлежит внутренности эллипса. Эта формулировка даёт дискретное приближение, ближайшее к непрерывной кривой, и, следовательно, силуэт наибольшей визуальной гладкости. Для малых диаметров, однако, результат может проявлять меньший пиксель-арт-характер: Евклидов круг $D = 5$ имеет лишь 13 блоков, тогда как Пороговый — 21.

Перенос в Minecraft. Евклидов критерий ближе всего к тому, что производит `//sphere` в WorldEdit. Чтобы построить сферу из булыжника $D = 16$ ($r = 8$) только с Евклидовой оболочкой, объём составляет $V \approx 2\,145$ блоков в заполненном режиме, около 804 блоков в тонком режиме (оболочка) — разница между сбором 34 паков булыжника ($34 \times 64 = 2\,176$) и 13 паков ($13 \times 64 = 832$).

5. Алгоритм Брезенхэма (целочисленная средняя точка)

Алгоритм Брезенхэма, опубликованный в 1965 году для черчения отрезков, был позднее расширен на окружности и обобщён Питтуэем и Каппелем на произвольные эллипсы. Его отличительное свойство — отказ от операций с плавающей запятой и извлечения квадратного корня: определение следующего пикселя использует исключительно **сложение и сравнение целых чисел** через *функцию решения*, которая оценивает, по какую сторону аналитической кривой находится средняя точка между двумя пикселями-кандидатами. Для строительства в Minecraft это алгоритм, дающий наиболее *узнаваемый* пиксель-арт-вид — тот же ступенчатый силуэт, который сообщество строит вручную с 2011 года.

5.1 Круговой случай

Для окружности целого радиуса R алгоритм стартует с $(x=0, y=R)$ с начальным параметром решения:

$$d_0 = 1 - R$$

На каждом шаге (в октанте от 0 до 45°):

$$\begin{cases} \text{если } d < 0 : & \text{следующий } (x+1, y), \quad d += 2x + 3 \\ \text{если } d \geq 0 : & \text{следующий } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

Обоснование. Функция d приближает на каждой итерации значение неявного уравнения окружности, вычисленное в **средней точке** между двумя пикселями-кандидатами:

$$F\left(x + 1, y - \frac{1}{2}\right) = (x + 1)^2 + \left(y - \frac{1}{2}\right)^2 - R^2$$

Знак d указывает, находится ли средняя точка внутри окружности (тогда продвигаемся только по горизонтали) или вне (тогда уменьшаем и вертикальную

координату). Восемь октантов окружности получаются применением симметрий диэдральной группы D_4 к вычисленному октанту, что в коде соответствует восьми вызовам `span(...)`. Результирующая трасса демонстрирует ступенчатый узор, характерный для дискретных приближений гладких кривых.

5.2 Эллиптический случай (две области)

Для эллипса с полуосями a, b алгоритм делит первый квадрант на **две области**, разделённые точкой, где касательная составляет 45° :

$$\text{Область I: } 2b^2x < 2a^2y \quad (|dy/dx| < 1), \quad \text{Область II: иначе}$$

Начальные параметры решения:

$$p_1 = b^2 - a^2b + \frac{a^2}{4},$$

$$p_2 = b^2\left(x + \frac{1}{2}\right)^2 + a^2(y - 1)^2 - a^2b^2.$$

На каждой итерации p обновляется предвычисленными приращениями (все целые после умножения на 4). Результат — минимально возможная арифметика на пиксель: *один тест знака и два сложения*.

Перенос в Minecraft. Для строителей в выживании силуэт Брезенхэма часто предпочтительнее Евклидова, так как его дискретные шаги легко считать вручную — пользователь знает, что следующий блок всегда либо *прямой*, либо *диагональный*, что сохраняет согласованную позицию руки при размещении. Круг Брезенхэма $D = 11$ имеет каноническую последовательность из 80 блоков, опубликованную в десятках руководств сообщества.

6. Пороговый алгоритм (покрытие углом)

Этот алгоритм спрашивает: *есть ли какой-нибудь угол ячейки внутри эллипса?* Для каждой ячейки (i, j) код находит угол, ближайший к центру формы (проекция центра на рёбра ячейки):

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{внутри} \iff \left(\frac{n_x - c_x}{r_x}\right)^2 + \left(\frac{n_y - c_y}{r_y}\right)^2 \leq 0,94$$

Значение $0,94$ — это *эмпирический порог*, чуть меньший $1,0$. Его назначение — устранить артефакт, известный как 1-пиксельный *касательный шип*, возникающий в четырёх кардинальных точках (С, Ю, В, З), где кривая касается целого ребра ячейки.

Среди трёх алгоритмов этот даёт силуэт наибольшей площади при том же D , поскольку условие включения наименее ограничительно. Для Minecraft это алгоритм, дающий самый *округлый* вид, но ценой больше блоков: $D = 16$ даёт 213 блоков оболочки по Пороговому против 200 по Брезенхэму и 192 по Евклидовому.

Перенос в Minecraft. Пороговый — выбор, когда постройка должна быть видна издали (например, кварцевый купол на храме), потому что чуть больший силуэт лучше читается через тени окружающей окклюзии. Компромисс: примерно 5–10% больше блоков на оболочку по сравнению с Евклидовым.

7. Режимы рендеринга: Заполненный, Тонкий, Толстый

Имея матрицу $f[j][i] = 1$ для внутренних ячеек, проект выводит три визуальных стиля через **дискретную топологию**:

7.1 Заполненный

Возвращает f без изменений. Все внутренние ячейки становятся блоками. Это то, что производит `//sphere stone 8`.

7.2 Тонкий (контур)

Ячейка принадлежит тонкому контуру, если она заполнена и имеет хотя бы одного ортогонального соседа (С, Ю, В, З) *вне* формы. В логической нотации:

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

Геометрически: это **дискретная граница** формы, дискретный аналог топологической границы $\partial F = F \cap \overline{F^c}$. В Minecraft это эквивалент `//hsphere` (полая сфера): материализуется только оболочка.

7.3 Толстый

Толстый режим добавляет к граничным ячейкам **диагональные мосты**: внутренние ячейки, имеющие одновременно горизонтального и вертикального граничного соседа. Это закрывает 1-пиксельные диагонали, которые тонкий режим оставляет открытыми, полезно, когда контур должен выглядеть стабильным в сцене (без отверстий под 45°) — особенно важно для куполов из стекла и льда.

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Перенос в Minecraft. Для стеклянного купола $D = 20$ (полого) толстый режим даёт *водонепроницаемую* оболочку из 608 блоков против ≈ 510 в тонком режиме — дополнительные ~ 100 блоков — это диагональные стежки, предотвращающие протечку окружающего света через диагональные щели.

8. Вычисление дискретной площади

Функция `area2D` просто **считает** ячейки, отмеченные в f . Это *считающая мера*, приближающая интеграл Римана от индикаторной функции эллипса:

$$\underbrace{\pi r_x r_y}_{\text{непрерывная площадь}} \longleftrightarrow \underbrace{\sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j)}_{\text{дискретная площадь}}$$

При больших D верно дискретная площадь/непрерывная площадь $\rightarrow 1$. При малых D разница между алгоритмами заметна: Пороговый склонен переоценивать; Евклидов остаётся близок к идеальной площади; Брезенхэм лежит где-то между ними.

Перенос в Minecraft. Чип счётчика блоков в UI Block Round показывает именно это число: 268 для $D = 8$, 904 для $D = 12$, 2145 для $D = 16$. Чип выражает счёт как N (*стаки* $\times 64$ + *остаток*) — так $2145 = 33 \times 64 + 33$, то есть «33 стака плюс 33 предмета», что и должен собрать пользователь.

9. От 2D к 3D: уравнение эллипсоида

В 3D основная форма — **эллипсоид** с центром в (c_x, c_y, c_z) и полуосями r_x, r_y, r_z . Его уравнение — естественное обобщение эллипса:

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

При $r_x = r_y = r_z$ возвращаемся к **сфере**. Вычисляя в центре каждого вокселя:

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{внутри} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Перенос в Minecraft. Эллипсоид с $W = 20$, $H = 10$, $D = 20$ даёт *приплюснутую сферу*, идеальную для подземного купола или крыши стадиона. Объём $V \approx 2094$ блоков (33 стака). Эквивалентная команда WorldEdit: `//ellipsoid stone 10 5 10`.

10. Воксельзация и вычисление объёма

Непрерывный объём эллипсоида — классический результат интегрального исчисления, получаемый интегрированием по дискам:

$$V = \frac{4}{3} \pi r_x r_y r_z$$

Дискретная версия перебирает каждый воксель в коробке $D_x \times D_y \times D_z$ и инкрементирует счётчик при $a_x^2 + a_y^2 + a_z^2 \leq 1$. Это алгоритм `voxelVolume`.

Оболочка и внутренность. В режиме *filled* проект показывает только воксели, имеющие хотя бы одного 6-соседа вне сохранённого множества (то есть воксели, видимые с внешней поверхности или с поверхности среза). В *thin* — только **поверхность** полного эллипсоида (толщиной 1 воксель). В выживании только видимые грани требуют отдельного текстурированного блока — внутренние воксели можно заполнить самым дешёвым доступным блоком (земля вместо алмаза) для экономии ресурсов.

Справочная таблица. Таблица ниже сводит канонические диаметры сфер, наиболее часто запрашиваемые в постройках сообщества Minecraft, в число блоков (V), паков по 64 ($P = \lceil V/64 \rceil$) и двойных сундуков на $54 \times 64 = 3456$ ячеек ($DC = \lceil V/3,456 \rceil$). Используйте эту таблицу для планирования сбора ресурсов перед строительством.

Диаметр	Блоки (V)	Паки ($\times 64$)	Двойные сундуки	Примечание
D=8	268	5	1	малый купол
D=12	904	15	1	средняя сфера
D=16	2145	34	1	вершина башни
D=20	4189	66	2	крыша стадиона
D=24	7238	114	3	большой купол
D=32	17156	269	5	мега-постройка

Объёмы в заполненном режиме, вычисленные по Евклидовому алгоритму для $D \in \{8, 12, 16, 20, 24, 32\}$. Значения в тонком режиме (оболочка) примерно $V_{\text{shell}} \approx 4\pi r^2$ блоков толщиной 1 воксель, то есть от 25% до 40% заполненного объёма в зависимости от D .

11. Геометрические срезы: плоскости и диагональный срез под 45°

Block Round позволяет резать тело **тремя плоскостями**. Каждый срез — это *линейное неравенство*, отбрасывающее воксели:

Ось X: $x \geq cut \Rightarrow$ отброшен

Ось Y: $y \geq cut \Rightarrow$ отброшен

Диагональ: $x + y \geq cut \Rightarrow$ отброшен

Плоскости X и Y перпендикулярны осям координат, с нормальми $(1, 0, 0)$ и $(0, 1, 0)$. *Диагональная плоскость* имеет нормаль $(1, 1, 0)/\sqrt{2}$ и режет под 45° в плоскости XY: математически это семейство плоскостей $x + y = k$. Поскольку максимум $x + y$ в коробке $D_x \times D_y$ равен $D_x + D_y$, ползунок диагонального среза имеет диапазон $D_x + D_y$, тогда как X и Y имеют диапазоны D_x и D_y соответственно. Срез **пропорциональный**: код хранит процент $cutPct$ и пересчитывает предел как

$$cut = cutPct \cdot \max_{\text{ось}}$$

так что 50% на одной оси остаётся 50%, когда пользователь переключает ось или меняет размер.

Перенос в Minecraft. Срез по оси Y на 50% для сферы даёт классический купол половинного объёма. Для $D = 16$: $V_{\text{full}} = 2\,145$ блоков, $V_{\text{dome}} \approx 1\,095$ блоков (≈ 18 паков). Диагональный срез на 50% даёт сечение *полу-чаша*, часто используемое для интерьеров фонтанов и сидений амфитеатров.

12. Толстый режим 3D: трёхмерализация через слои

Для режима *thick* в 3D проект применяет 2D-толстый алгоритм ко **всем срезам** по трём осям: XY для каждого z , XZ для каждого y , YZ для каждого x . Затем берёт *объединение* результатов. Геометрическая мотивация — герметичность: минимальное множество вокселей, закрывающих все диагонали без удвоения толщины оболочки. Формально, при $S_z(z)$ — срез XY на высоте z и T — 2D-толстый оператор:

$$\text{thick3D} = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

Результат затем пересекается с множеством, сохранённым срезом. Лежащая в основе теория — **цифровая топология**: толстый оператор гарантирует 26-связность оболочки (соседи, включая диагонали), полезную для строительства в Minecraft, где диагональные воксели не соприкасаются гранями — свет, вода и существа проходят через диагональные щели, как если бы стены не было.

Перенос в Minecraft. Для подводного стеклянного купола над океаническим монументом в выживании толстый режим обязателен: диагональные мосты предотвращают просачивание воды через диагональные трещины, оставленные тонким режимом. Стоимость: купол $D = 20$ в толстом режиме требует ≈ 380 блоков стекла против ≈ 320 в тонком — на 19% больше материала в обмен на полную герметичность.

13. 3D-камера: сферические координаты

3D-камера обращается вокруг объекта на расстоянии r с двумя углами — θ (азимут, в горизонтальной плоскости) и φ (полярный, от вертикальной оси). Это **физическое соглашение** сферических координат, и преобразование в декартовы (формат, ожидаемый *three.js*):

$$\begin{aligned}x &= r \sin(\varphi) \cos(\theta), \\y &= r \cos(\varphi), \\z &= r \sin(\varphi) \sin(\theta).\end{aligned}$$

Начальное положение: $\theta = \pi/4$ (45°, изометрическая перспектива) и $\varphi = \pi/3$ (60° от северного полюса, чуть выше экватора). Перетаскивание мыши инкрементирует θ и φ напрямую; колесо/щипок инкрементирует r . Простота этой параметризации позволяет свободное вращение без кватернионов или явных матриц.

14. Авто-зум: ограничивающая сфера и FOV

Когда пользователь меняет размер фигуры или сбрасывает камеру, проект пересчитывает **идеальное расстояние** камеры, чтобы объект помещался в вьюпорт под любым углом. Идея — заключить объект в *сферу радиуса R* (половина диагонали AABB, выровненной по осям ограничивающей коробки):

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

Сфера имеет **инвариантную при вращении проекцию**, поэтому если она помещается в вьюпорт в одной ориентации, она помещается во всех. При вертикальном FOV камеры ($fov_V = 35^\circ$ в радианах) расстояние, необходимое для кадрирования сферы радиуса R :

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

Горизонтальный FOV получается из соотношения сторон холста:

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

Конечное расстояние — максимум $dist_V$ и $dist_H$, умноженный на 1,20 (20% визуального отступа). Когда фигура обрезана, код сжимает D_x или D_y до оставшегося размера. Когда пасхалка дуба или крипера активна, ограничивающая коробка расширяется вверх для включения высоты сущности.

15. Затенение и умножение текстуры

Три 3D-стиля (*Classic, Smooth, Blocks*) различаются **мультипликативными факторами**, применяемыми к трём видимым граням каждого вокселя (верх, освещённая сторона, тёмная сторона). Функция `shadeHex(hex, factor)` парсит `hex` в (R, G, B) и умножает каждый канал с зажимом в $[0, 255]$ — и результат становится оттенком, применяемым поверх сырой текстуры блока:

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

Это *линейное приближение* идеальной ламбертовой функции освещения,

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \text{текстура},$$

где три фактора соответствуют косинусу угла между нормалью каждой грани и фиксированным направлением света. В *Smooth* факторы близки (схожие грани); в *Classic* факторы различаются больше (сильный контраст, вид ванильного Minecraft). *Blocks* дополнительно вводит геометрический отступ в воксели — постоянный перенос каждой грани внутрь, чтобы выявлять межвоксельные границы даже когда соседние воксели несут одну и ту же текстуру. Для излучающих блоков (светокамень, морской фонарь, грибной светильник, магма, плачущий обсидиан) ламбертов член заменяется константой $I = \text{emissive_factor} \cdot \text{texture}$, масштабированной до уровня света блока в MC.

16. Переход: от непрерывной математики к строительству в игре

Пятнадцать разделов выше описывают *непрерывный-к-дискретному* конвейер, который Block Round разделяет со своим родственным проектом Pixel Round: от неявного уравнения эллипса до камеры, которая кадрирует результат. Оставшиеся шесть разделов описывают то, что **специфично** для Block Round — дополнительный математический слой, вводимый тем фактом, что отображаемые ячейки — это не абстрактные пиксели, а *блоки Minecraft*, каждый с шестью текстурированными гранями, весом в инвентаре и стоимостью игрового времени для сбора и размещения.

Это не косметические соображения. Решение визуализировать с текстурами блоков меняет операционную проблему пользователя: вместо *экспорта PNG*, пользователь должен *перевести силуэт в план размещения*. Математическое содержание следующих разделов касается именно этого перевода: арифметика инвентаря, оптимизация подсчёта граней, разложение слой за слоем и использование симметрии для сокращения стоимости от N размещений до $N/8 + 7$ вызовов команд.

17. Арифметика инвентаря Minecraft

Игрок Minecraft несёт предметы в *инвентаре* из 36 ячеек, организованном как сетка 9×4 . Каждая ячейка вмещает до 64 предметов одного типа (один *стак* или *пак*). Игрок также имеет доступ к *хранилищу*: одиночный сундук — 27 ячеек, двойной сундук — 54 ячейки. Арифметика, переводящая целевое число блоков в план сбора, есть, следовательно, задача *деления с округлением вверх*.

17.1 Одна ячейка, один стак, полный пак

Стак — это элементарная единица инвентарного учёта. Каждый стак содержит не более 64 предметов. Перевод числа блоков N в паки:

17.2 Хранение: сундуки, двойные сундуки, шалкеры

Чтобы вычислить, сколько *двойных сундуков* требует постройка, разделите число блоков на $54 \text{ ячейки} \times 64 \text{ предмета/ячейка} = 3\,456$ предметов на двойной сундук:

$$\text{паки : } N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil \quad \text{двойные сундуки : } N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$$

Одиночный сундук вмещает $27 \times 64 = 1\,728$ предметов. *Шалкер* (переносной контейнер) вмещает $27 \times 64 = 1\,728$ предметов, но сам может храниться *внутри* ячейки сундука, поэтому двойной сундук, наполненный 54 шалкерами, может вместить до $54 \times 1\,728 = 93\,312$ предметов. Для мега-построек релевантная единица ёмкости — *двойной сундук, нагруженный шалкерами*.

17.3 Справочная таблица: канонические сферы

Таблица ниже преобразует канонические объёмы заполненных сфер в планы сбора. Последний столбец предлагает контекст постройки для каждого диаметра:

Диаметр	V (блоки)	Паки	Хранение	Типичная постройка
D=8	268	5	1 дс	крыша хижины, купол колодца
D=12	904	15	1 дс	верх дозорной башни
D=16	2145	34	1 дс	колокол деревни, купол библиотеки
D=20	4189	66	2 дс	купол обсерватории
D=24	7238	114	3 дс	купол храма
D=32	17156	269	5 дс	купол собора, арена мирового босса

Представление *стаки и остаток* $N = q \cdot 64 + r$, показываемое чипом инфо Block Round, отражает, как инвентарь выживания отображает частичные стаки: игрок видит q полных ячеек плюс одну ячейку, показывающую r . Для $D = 16$ чип печатает «2 145 (33 × 64 + 33)», говоря игроку заполнить 33 ячейки полностью плюс одну ячейку с 33 предметами — ровно 34 ячейки.

18. Подсветка обводки и подсчёт открытых граней

Block Round по умолчанию рендерит чёрную *подсветку обводки* (highlight overlay) на рёбрах каждой видимой грани вокселя. Визуально это совпадает со стилем «F3+G» границ чанков игрового отладочного overlay, масштабированным до уровня одного вокселя. Математически overlay реализует визуализацию **подсчёта открытых граней**: число, которое используют строители выживания для проверки прогресса блок за блоком.

18.1 Что считается открытой гранью

Грань вокселя $\mathbf{v}$, разделяемая с соседом $\mathbf{n}$, *открыта* тогда и только тогда, когда $\mathbf{n} \notin V_{\text{solid}}$ (сосед пуст). Каждый твёрдый воксель вкладывает от 0 (полностью погребён) до 6 (изолирован) граней. Общее число открытых граней F_{exp} — дискретный аналог *площади поверхности* фигуры.

18.2 Формула обнаружения границы

Воксель находится на границе, если хотя бы один из его шести соседей по граням находится вне твёрдого тела:

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

Это тот же оператор границы, что и тонкий 2D-алгоритм, расширенный до 3D, и overlay просто рисует линии куб-рёбер каждого граничного вокселя. Для стекла и льда значение по умолчанию ОТКЛ.; для булыжника, камня, алмаза и других непрозрачных блоков значение по умолчанию ВКЛ.

18.3 Подсчёт открытых граней как эвристика выживания

Для строителей выживания релевантным числом является не общий объём V , а *количество открытых граней* F_{exp} , потому что это то, что видно снаружи. Сумма F_{exp} по всем твёрдым вокселям равна шести количеству граничных вокселей минус удвоенное число граничных смежностей между граничными воксе-

лями:

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

Для сферы $D = 16$ (заполненный режим), $V = 2\,145$, $V_{\text{граница}} \approx 432$ (оболочка), $F_{\text{exp}} \approx 1\,184$ видимых граней — постройка нуждается лишь в около 432 видимых блоках; остальные 1 713 блоков — внутренность, которую можно заполнить самым дешёвым доступным блоком (земля, булыжник). Overlay позволяет строителю мгновенно заметить пропущенный блок в оболочке.

19. Послойное строительство (горизонтальное разложение)

В Minecraft строительство идёт *горизонтальными слоями*: игрок стоит на слое, размещает блоки слоя выше, затем поднимается на один блок (прыжок + размещение временного строительного блока, или использование элитр). Поэтому 3D-задачу эллипсоида лучше всего понимать как *стопку 2D-эллипсов*, по одному на слой:

$$\text{слой}(k) = \{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \}$$

Для каждого целого слоя k от $-r_z$ до $+r_z$ поперечное сечение само есть эллипс с уменьшенными полуосями $r_x(k)$ и $r_y(k)$:

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

Это означает, что 3D-задача сводится к *вычислению 2D-алгоритма предыдущих разделов, по одному слою за раз*, и складыванию результатов. Когнитивно это огромное упрощение: вместо ментального отслеживания трёх координат строитель отслеживает две координаты на слой плюс индекс слоя.

Перенос в Minecraft. Для сферы $D = 16$ ($r_z = 8$) экваториальный слой ($k = 0$) — это полный круг Брезенхэма $D = 16$ (≈ 200 блоков). Следующий слой выше ($k = 1$) — круг $D = 15,97$. К $k = 4$ слой — круг $D = 13,86$ (около 148 блоков), а к $k = 7$ он сжимается до круга $D = 7,48$ (около 43 блоков). Полюсы ($k = \pm 8$) — одноблочный колпак.

Чип инфо Block Round открывает этот послойный счёт в 3D-режиме, когда пользователь активирует угловую кнопку центральных направляющих (клавиша с). Горизонтальные срезы рисуются как полупрозрачный жёлтый крест на экваторе и на каждом делении $r_z/2$.

20. Оптимизация через октаэдрическую симметрию (O_h)

Сфера, центрированная в начале координат, инвариантна под действием **октаэдрической группы** O_h порядка 48. Воксельзация сферы сохраняет подгруппу порядка 48, порождённую тремя отражениями координат и тремя вращениями $\pi/2$ вокруг каждой оси. Для практических целей строительства релевантной подгруппой является группа отражений порядка 8 *октанта* $\mathbb{Z}_2^3$, порождённая $x \mapsto -x, y \mapsto -y, z \mapsto -z$. Множество вокселей в одном октанте определяет полную сферу:

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{сокращение} = \frac{N - N/8}{N} = 87,5\%$$

Это наибольшая математическая оптимизация, доступная строителю Minecraft. Вместо ручного размещения N блоков строитель размещает $N/8$ блоков в положительном октанте и затем выдаёт семь команд `/clone` (ваниль) или `//copy + //paste` (WorldEdit) с соответствующими отражениями.

20.1 Конкретная последовательность команд

Для сферы $D = 24$ ($r = 12, V = 7\,238$ блоков), центрированной в мировых координатах $(0, 72, 0)$, сначала постройте положительный октант (область $+x, +y, +z$, ≈ 905 блоков), затем выдайте:

```

/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (отраж.  $-x$ )
/clone 0 72 0 12 84 12 0 60 0 mode:masked (отраж.  $-y$ )
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (отраж.  $-z$ )
/clone -12 72 0 -1 84 12 mode:masked (октант  $-x, -y$ )
/clone 0 60 -12 12 71 -1 mode:masked (октант  $-y, -z$ )
/clone -12 72 -12 -1 84 -1 mode:masked (октант  $-x, -z$ )
/clone -12 60 -12 -1 71 -1 mode:masked (октант  $-x, -y, -z$ )

```

Каждый `/clone` стоит около 50 мс серверного времени, так что семь отражений завершаются менее чем за полсекунды. Ручное размещение 905 блоков октанта занимает около 15 мин в выживании или 45 с в творческом с кистью. Полное размещение 7 238 блоков (без симметрии) заняло бы ≈ 2 ч в выживании или 6 мин в творческом.

20.2 Сравнение временных затрат

Пусть T_{block} — время размещения одного блока (обычно 1–2 с в выживании, 0,1–0,5 с в творческом), T_{clone} — время одной команды клонирования (около 50 мс). Общее время постройки с симметрией и без неё:

$$T_{\text{octant}} + 7 T_{\text{clone}} \ll T_{\text{full}}$$

Для $N = 7\,238$ в выживании ($T_{\text{block}} = 2$ с, $T_{\text{clone}} = 0,05$ с): полностью = 14 476 с ≈ 4 ч 1 мин; октант + клоны = 1 810 с + 0,35 с ≈ 30 мин. Ускорение в 8 раз точно соответствует порядку подгруппы отражений. Полная октаэдрическая группа O_h порядка 48 в принципе может сократить дальше (в 48 раз), но использование диагональных вращений требует области строительства, повернутой на 45° , что редко оправдывает стоимость настройки в ванильном Minecraft.

21. Текстуры блоков и визуальная композиция

Блок Minecraft — это не плоская окрашенная ячейка, а куб с шестью текстурированными гранями. Block Round рендерит каждый воксель его шестью каноническими гранями текстур, выбираемых из ванильного набора ресурсов. Математически операция текстурирования — это *UV-разметка*: каждая грань единичного куба параметризуется координатами $[0, 1]^2$, и соответствующая текстура выбирается в этих UV для получения рендеренных пикселей.

21.1 Шесть граней и UV-разметка

Для каждой грани вокселя рендерер обращается к модели блока:

$$\text{грань} \in \{ \text{верх, бок, низ} \} \quad UV : [0, 1]^2 \rightarrow \text{pixels}$$

Большинство блоков (камень, булыжник, алмазный блок, дубовые доски) используют *одну и ту же текстуру на всех шести гранях*. Многогранные блоки (блок травы, тыква, тюк сена, кварцевая колонна, все древесные брёвна, верстак, печь, книжный шкаф, TNT) используют *разные текстуры* наверху, по бокам и внизу: блок травы имеет зелёный верх, бок с травой-землёй и чисто земляной низ; дубовое бревно имеет древесные кольца наверху и низу и кору на четырёх боках. Принципиально, что *математика того, какие воксели размещать*, не зависит от используемой текстуры — силуэт алмазной сферы и булыжной сферы одного диаметра идентичен. Решение о текстуре — это *чисто эстетический* слой, применяемый после завершения геометрического алгоритма.

21.2 Семейства блоков и тона

Для строителя выживания, выбирающего среди десятков доступных блоков, релевантный вопрос — не индекс текстуры, а *тональное семейство*: светлый/тёмный, тёплый/холодный, однородный/узорчатый. В таблице ниже классифицированы наиболее распространённые блоки в постройках Block Round:

Блок	Семейство	Тон	Типичное использование
cobblestone	камень	серый	деревенские стены, подземелья
stone	камень	серый	чистые стены, пьедесталы
oak_planks	дерево	бежевый	полы, внутренняя отделка
quartz_block	кварц	белый	храмы, современные купола
sandstone	песок	бежевый	пустыни, пирамиды
deepslate	камень	тёмный	подземелья, акценты

21.3 Градиенты через смешение блоков

Хотя один тип блока даёт один тон, *градиенты* можно получить чередованием двух или трёх типов блоков на слой. Классическая техника Minecraft — градиент *камень-к-кварцу* на куполе: нижние слои — гладкий камень, экваториальные слои — шумная смесь гладкого камня и диорита, верхние слои — кварц. Каждый слой остаётся вычисленным тем же алгоритмом Block Round; меняется поиск текстуры на воксель. Опция блока *Random* в Block Round реализует одното такое процедурное смешивание по умолчанию (верх травы, под ним земля,

разреженные руды в середине, глубокий сланец близ коренной породы). Математика та же, что и в послойном разложении из раздела 19; меняется только «тег блока» на воксель.

22. Заключение

Настоящий документ систематически описал математические основы, используемые в Block Round. В приблизительно тысяче строк JavaScript-кода плюс поддерживающие конвейеры текстур и аудио проект интегрирует вклады из нескольких теоретических областей:

- **Аналитическая геометрия:** неявные уравнения эллипса и эллипсоида как первичный критерий принадлежности.
- **Классические алгоритмы растеризации:** Брезенхэм в формулировке средней точки, с расширением Питтуэя/Каппеля для эллипсов.
- **Цифровая топология:** операторы дискретной границы, концепции 4-, 6- и 26-связности, композиция через срезы и подсчёт открытых граней.
- **Интегральное исчисление:** объём эллипсоида как тройной интеграл и его дискретный аналог (считающая мера по вокселям).
- **Линейная алгебра:** секущие плоскости, определяемые линейными неравенствами, и трёхмерная перспективная проекция.
- **Тригонометрия:** параметризация камеры в сферических координатах и автоматическая настройка расстояния в зависимости от FOV.
- **Комбинаторная арифметика инвентаря:** учёт с округлением вверх для паков по 64 и двойных сундуков на 3 456 предметов, с представлением стаки-и-остаток, отображаемым чипом инфо.
- **Симметрия групп:** использование октаэдрической группы O_h и её подгруппы отражений порядка $8 \mathbb{Z}_2^3$ для сокращения стоимости строительства в 8 раз через `/clone`.

Центральное наблюдение, которое позволяет сформулировать исследование, следующее: на дискретной решётке **не существует единого правильного представления** непрерывной кривой. Каждый алгоритм, представленный в этом документе, соответствует отдельному математическому определению критерия включения ячеек, и каждое определение производит силуэт с собственными формальными свойствами — гладкость в Евклидовом алгоритме, ступенча-

тый узор Брезенхэма, большее покрытие в критерии угловой покрываемости. Поэтому выбор алгоритма — это техническое решение, которое должно учитывать намеченное визуальное представление, желаемые метрические характеристики получаемой формы и — конкретно для Block Round — стоимость инвентаря и времени для физического построения формы в игре.

Block Round занимает небольшую, но математически богатую нишу в экосистеме инструментов Minecraft: он находится между чисто геометрическими инструментами (WorldEdit, Litematica) и чисто визуальными (наборы ресурсов, шейдеры), предлагая явный, выводимый и воспроизводимый мост от непрерывного уравнения к дискретному плану размещения. Родственный проект Pixel Round предоставляет те же алгоритмы без слоя Minecraft, подходящий для традиционных приложений пиксельной и воксельной графики. Вместе оба проекта иллюстрируют, как один и тот же непрерывный-к-дискретному математический конвейер поддерживает радикально разные художественные и конструктивные рабочие процессы.

Приложение А: справочные таблицы и пошаговый пример

Это приложение собирает наиболее часто используемые числовые и командные справочники для пользователей Block Round. Данные ниже консолидируют таблицы из отдельных разделов и пошаговые примеры, разбросанные по документу, организованные так, чтобы приложение функционировало как одностраничный справочник для планирования постройки, синтаксиса команд и сквозного пошагового примера.

А.1 Полная справка по сферам (заполненная, полая, паки)

Таблица ниже расширяет канонические таблицы сфер из разделов 10 и 17, добавляя вариант полой оболочки (*тонкий режим*) рядом с заполненным объёмом. Подсчёт полой оболочки предполагает один воксель толщины и вычисляется по Евклидовому алгоритму; толстый режим добавляет примерно 10–15% дополнительных блоков для герметизации диагональных щелей, как описано в разделе 12.

D	Заполн. (V)	Полая (обол.)	Паки запол.	Паки полая
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

Для строителя в режиме выживания обычно релевантна колонка *полая*: только видимая оболочка нуждается в специальном текстурированном блоке, а внутренность (если она вообще нужна) может быть заполнена самым дешёвым материалом. Колонка паки-заполненная даёт общий размер запаса для полностью заполненной сферы; паки-полая — реалистичный минимум для полого купола.

А.2 Справочник команд WorldEdit / vanilla

Следующая таблица суммирует команды WorldEdit и vanilla-Minecraft, наиболее значимые для построения выходов Block Round в игре. Команды WorldEdit предполагают, что игрок держит палочку (по умолчанию деревянный топор) и выделил область; vanilla-команды работают без модов, но требуют аргументов координат.

Команда	Мод / происхождение	Результат
<code>//sphere stone 8</code>	WorldEdit	заполн. сфера $r=8$ (Евклид)
<code>//hsphere stone 8</code>	WorldEdit	полая сфера $r=8$ (только обол.)
<code>//cyl stone 8 16</code>	WorldEdit	цилиндр $r=8, h=16$
<code>//ellipsoid st. 10 5 10</code>	WorldEdit	эллипсоид $10 \times 5 \times 10$
<code>/fill ... stone</code>	vanilla	заполнить коробку одним блоком
<code>/clone ... mode:masked</code>	vanilla	копировать область в новое место

Для пользователей Litematica рабочий процесс другой: вместо выполнения команд файл схематики (Block Round экспортирует `.schem`, формат Sponge v2) загружается в мод Litematica, который затем отображает полупрозрачный призрак фигуры, который игрок размещает блок за блоком. Это ближайший эквивалент режима выживания к креативному опыту WorldEdit.

А.3 Пошаговый пример: купол $D=20$ из кварца на храме

Как полный пошаговый пример, предположим, строитель хочет купол из кварца $D = 20$ на вершине существующего каменного храма. Вершина храма 11×11 блоков, центрирована в $(0, 80, 0)$ в мировых координатах. Купол — верхняя половина сферы радиуса $r = 10$, обрезанная по плоскости $Y = 0$. План строительства:

- Планирование с Block Round.** Открыть приложение, режим 3D, Сфера, размер $D = 20$, ось среза Y на 50%. Выбрать кварцевый блок из подборщика. Чип инфо сообщает $V_{\text{купол}} \approx 2\,126$ блоков (≈ 34 пака, 1 двойной сундук с 1 330 предметами, оставшимися во втором).
- Собрать ресурсы.** 34 пака кварцевого блока = $\lceil 2\,126/4 \rceil = 532$ кварцевых блока $\times 4$ незер-кварца каждый = 2 128 незер-кварца. Плавить или торговать по необходимости.
- Выбрать алгоритм.** Для кварцевого купола, видимого снизу, Евклидов алгоритм даёт самый гладкий силуэт; для купола на удалённой горе Порого-

вый алгоритм читается чище. Block Round по умолчанию использует Евклидов.

4. **Позиционировать купол.** Плоская грань купола должна совпасть с вершиной храма. Ограничивающая коробка купола $20 \times 10 \times 20$ блоков; разместите его центр в $(0, 80, 0)$, чтобы плоская (срезанная) грань лежала в $y = 80$, а вершина купола — в $y = 89$.
5. **Строить по симметрии.** Постройте квадрант $+x, +z$ купола (≈ 530 блоков). Выдайте `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` для зеркального отражения по x ; затем `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` для отражения по z . Общее истёкшее время: ~ 10 минут в выживании.
6. **Полировка.** Добавьте морские фонари на вершине и в четырёх кардинальных точках экватора для эмиссивного освещения (Block Round рендерит их с их эмиссивной картой на уровне света MC 15). Опционально переключите оверлей рёбер (клавиша G), чтобы проверить оболочку купола на отсутствие пробелов перед размещением последних блоков.

Математика каждого шага — это именно то, что описывают предыдущие двадцать два раздела: неявное уравнение эллипсоида, Евклидова воксельзация, срез по оси Y, редукция октаэдрической симметрии через `/clone`, арифметика инвентаря для перевода объёма в паки и сундуки. Поэтому приложение — это не новая теория, а напоминание о том, что каждая теоретическая концепция этого документа сопоставляется с единым конкретным решением в рабочем процессе планирования постройки.

А.4 Справочник клавиатурных сокращений

Приложение Block Round предоставляет небольшой набор клавиатурных сокращений, отражающих кнопки в углах холста. Они перечислены ниже для быстрой справки; сокращения глобальны (не требуется фокус ни на одном поле ввода) и работают как в 2D, так и в 3D режимах, если не указано иное.

Клавиша	Переключает	Примечания
G	Наложение рёбер	чёрный контур на каждой грани вокселя
C	Центральные направляющие	полупрозрачный жёлтый крест на осях
D	Скачать PNG	текущий холст как изображение PNG
I	Чип инфо	счёт блоков и разбивка по пакам
M	Переключение 2D / 3D	плоский и воксельный режим
S	Звук вкл/выкл	окружающие звуки размещения блоков
T	Тема день / ночь	затемняет фон, держит плитки читаемыми
Ctrl+Z / Y	Отменить / повторить	проходит историю изменений фигуры

Сокращения G (сетка) и C (центральные направляющие) особенно полезны при валидации постройки: G показывает, на месте ли каждый воксель оболочки (пропущенный блок ломает паттерн), а C подтверждает, что срезы правильно центрированы на осях фигуры.

Для Ctrl+Z отмены: Block Round проходит каждую правку, изменяющую фигуру (ползунки, режим рендера, алгоритм, форма, ось, блок, переключение режима), но намеренно исключает чисто визуальные переключения (камера, наложение рёбер, центральный крест, тема, звук). Это разделение наполняет стек отмены правками, которые пользователь, вероятно, захочет отменить.

А.5 Глоссарий ключевых терминов

Быстрая справка по техническим терминам, повторяющимся в этом документе. Каждая запись кратко резюмирует концепцию и указывает раздел, в котором она развивается подробно.

- **Воксель** — единичный куб в 3D, аналог пикселя в 2D. Каждый блок Minecraft - это один воксель. См. раздел 2.
- **Пак (стак)** — в Minecraft - стак из 64 одинаковых предметов, занимающий одну ячейку инвентаря. См. раздел 17.
- **Двойной сундук** — два соседних сундука, объединённых в контейнер на 54 ячейки. Вмещает до 3 456 предметов. См. раздел 17.
- **Шалкер-коробка** — переносной контейнер на 27 ячеек, который сам может храниться внутри ячейки сундука. См. раздел 17.
- **Оболочка** — внешний толщиной в один воксель слой сплошной фигуры. Релевантная поверхность для строителей в выживании. См. разделы 7 и 18.
- **Октант** — одна из восьми областей трёхмерного пространства, получаемых

пересечением трёх координатных полупространств. Используется в оптимизации по симметрии. См. раздел 20.

- **UV-разметка** — 2D-параметризация 3D-границ, используемая для наложения текстурного изображения на каждую грань вокселя. См. раздел 21.
- **Брезенхэм** — инкрементный алгоритм растеризации 1965 года, использующий только целочисленную арифметику. См. раздел 5.
- **Пороговый** — критерий растеризации, включающий ячейку, если любой её угол находится внутри эллипса. См. раздел 6.
- **Евклидов** — критерий растеризации, включающий ячейку, если её центр находится внутри эллипса. См. раздел 4.
- **Пасхалка** — скрытая функция, активируемая определённым вводом. В Block Round: дуб и крипер, оба на значении ползунка 15.
- **Схематика** — сериализованный файл структуры (Sponge Schematic v2, `.schem`), загружаемый WorldEdit, Litematica или MCEdit.

Этот глоссарий намеренно остаётся короче полного математического словаря. Читатели, ищущие более глубокого охвата лежащей в основе математики (цифровая топология, интегральное исчисление, теория групп и т.д.), отсылаются к соответствующим разделам данного документа.

А.6 Перспективы и пределы модели

Модель Block Round, представленная в этом документе, охватывает статическую вокселизацию эллипсов, эллипсоидов и их срезов, а также практические соображения, вводимые слоем Minecraft. Она не рассматривает динамические задачи — движущиеся фигуры, анимированные воксельные переходы, взаимодействие с симуляцией жидкости — поскольку они требуют непрерывно-временной структуры за пределами дискретной растеризации. Естественным расширением является *анимация на уровне схематики*: последовательность статических вокселизаций, соединённых интерполяцией, которую Block Round уже может экспортировать как поток `.schem`, читаемый скриптами Litematica.

Другое открытое направление — *точная сохраняющая объём вокселизация*: текущий дискретный счёт V_{disc} сходится к непрерывному объёму V_{cont} лишь асимптотически. Для приложений, где точный счёт должен соответствовать непрерывной формуле (например, рендеринговые соревнования, математическое искусство со строгим бюджетом блоков), необходим дополнительный слой уточнения — например, лёгкая регулировка радиуса так, чтобы дискретный счёт

попадал в целевое значение. Block Round в настоящее время не реализует это уточнение, но лежащая в основе математика проста: численно решить $V_{\text{disc}}(r) = V_{\text{target}}$ относительно r .

Наконец, *использование симметрии более высокого порядка*: настоящий документ покрывает подгруппу отражений порядка 8 группы O_h , но полная группа порядка 48 включает вращения, которые в принципе позволили бы ускорение в $\times 48$ раз. Практическое препятствие в том, что внутриигровые команды работают в выровненных по осям областях, а использование вращательных симметрий требует области постройки, повернутой на 45° . Серверные моды, которые внутренне поворачивают выделения (например, FAWE), могут в принципе этого достичь, но основной инструментарий пока не предоставляет это. Эта-лонная реализация, демонстрирующая ускорение в $\times 48$ на ванильном сервере, стала бы естественным продолжением настоящей работы.

А.7 Дополнительные ссылки и внешние ресурсы

Математические концепции, развитые в этом документе, имеют собственные хорошо устоявшиеся литературы. Читателям, ищущим первоисточники или более глубокий алгоритмический фон, рекомендуются следующие точки входа:

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 — оригинальная статья о растеризации целочисленной арифметикой, расширенная в данном документе.
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 — обобщает Брезенхэма на произвольные конические сечения.
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 — эллиптическая формулировка с двумя областями, использованная в разделе 5.
- R. Klette и A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 — стандартная справка по цифровой топологии и операторам границы из разделов 7 и 18.
- Документация WorldEdit, enginehub.org/worldedit/ — официальный справочник по командам `//sphere`, `//hsphere`, `//ellipsoid`, `//copy`, используемым в данном документе.
- Спецификация Sponge Schematic v2, github.com/SpongePowered/Schematic-Specification — формат файла, который Block Round экспортирует как `.schem`.

Восемь теоретических областей, перечисленных в разделе 22, имеют каждая по отдельности учебниковые трактовки, далеко выходящие за пределы того, что один технический документ может суммировать. Вклад Block Round не в самой теории, а в конкретном синтезе: применение классической растеризации к текстурированной воксельной обстановке Minecraft с явными соображениями инвентаря и стоимости симметрии, которые обычно опускают чисто геометрические инструменты.

Заключительное замечание о воспроизводимости: весь конвейер этого документа — скрипт сборки, переводы, шаблон LaTeX, локализованные таблицы — содержится в директории `docs_math/` репозитория проекта. Повторный запуск `python build.py` на системе с XeLaTeX/MiKTeX точно воспроизводит этот PDF на любом из девяти поддерживаемых языков. Родственный проект Pixel Round следует той же структуре, позволяя прямое сравнение текстурированных и нетекстурированных вариантов бок о бок.

Конец документа

Block Round поддерживается Vinícius Rodrigues de Souza. Родственный проект Pixel Round доступен по адресу github.com/ViniSouza128/pixel-round; текущий репозиторий Block Round находится по адресу github.com/ViniSouza128/block-round.

Комментарии, исправления и улучшения переводов приветствуются через issue tracker репозитория проекта. Особенно ценится вычитка носителями языка для восьми неанглийских локалей, так как локализации были подготовлены со значительным пересмотром, но технико-математическая терминология выигрывает от исправлений носителей языка.